Tutorial 5: Train your own LLMs

Course Name: CSC6052/5051/4100/DDA6307/MDS5110 Natural Language Processing

This notebook guide provides a comprehensive overview of using the `transformers` Python package to efficiently train a custom model. It covers the following techniques:

1. Load Model, Tokenizer and Template for Chat Model.
2. Process Data for Training.
3. Train Model with Qlora.
4. Evaluate Model's performance.
5. Save and Deploy Trained Model.

Preliminary Preparation

Before proceeding with model training, ensure your environment is properly configured by following these steps:

1. Install the necessary Python packages.
2. Import the required libraries.

```
!pip install -q h5py typing-extensions wheel
!pip install -q -U bitsandbytes
!pip install -q -U git+https://github.com/huggingface/transformers.git
!pip install -q -U git+https://github.com/huggingface/peft.git
!pip install -q -U git+https://github.com/huggingface/accelerate.git
!pip install -q datasets
!pip check
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
No broken requirements found.
```

```
!nvidia-smi
```

```
Sun Mar 30 15:31:03 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L4                      Off |   00000000:00:03.0 Off |                    0 |
| N/A   35C    P8             11W /   72W |       0MiB /  23034MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Load Pre-trained model and tokenizer

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
model_id = "Qwen/Qwen2.5-7B-Instruct"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True, # Activate nested quantization for 4-bit base models (double quantization)
    bnb_4bit_quant_type="nf4", # Quantization type (fp4 or nf4), According to QLoRA paper, for training 4-bit ba
    bnb_4bit_compute_dtype=torch.bfloat16
)
model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config, device_map={"":0})

tokenizer = AutoTokenizer.from_pretrained(model_id)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/setti
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

config.json: 100% 663/663 [00:00<00:00, 76.0kB/s]

model.safetensors.index.json: 100% 27.8k/27.8k [00:00<00:00, 3.14MB/s]

Fetching 4 files: 100% 4/4 [00:49<00:00, 49.96s/it]

model-00002-of-00004.safetensors: 100% 3.86G/3.86G [00:48<00:00, 91.7MB/s]

model-00001-of-00004.safetensors: 100% 3.95G/3.95G [00:49<00:00, 200MB/s]

model-00004-of-00004.safetensors: 100% 3.56G/3.56G [00:45<00:00, 80.3MB/s]

model-00003-of-00004.safetensors: 100% 3.86G/3.86G [00:49<00:00, 150MB/s]

```
Sliding Window Attention is enabled but not implemented for `sdpa`; unexpected results may be encountered.
```

Loading checkpoint shards: 100% 4/4 [00:20<00:00, 4.80s/it]

generation_config.json: 100% 243/243 [00:00<00:00, 28.9kB/s]

tokenizer_config.json: 100% 7.30k/7.30k [00:00<00:00, 921kB/s]

vocab.json: 100% 2.78M/2.78M [00:00<00:00, 6.59MB/s]

merges.txt: 100% 1.67M/1.67M [00:00<00:00, 7.30MB/s]

tokenizer.json: 100% 7.03M/7.03M [00:00<00:00, 8.10MB/s]

Preprocess the quantized model for training

```
from peft import prepare_model_for_kbit_training

model.gradient_checkpointing_enable()
model = prepare_model_for_kbit_training(model)
```

```
from peft import LoraConfig, get_peft_model

# You can try differnt parameter-effient strategy for model trianing, for more info, please check https://github
config = LoraConfig(
    r=8,
    lora_alpha=8,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, config)
```

Chat Template Usage

```
from jinja2 import Template
template = Template(tokenizer.chat_template)
```

```
message = "Please introduce yourself"
print(f"message:\n{message}\n")
message_send_to_model=template.render(messages=[{"role": "user", "content": message}],bos_token=tokenizer.bos_to
print(f"message_send_to_model:\n{message_send_to_model}")
```

```
message:
Please introduce yourself

message_send_to_model:
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Please introduce yourself<|im_end|>
<|im_start|>assistant
```

```
template = Template(tokenizer.chat_template)
@torch.no_grad()
def generate(prompt):
    modelInput=template.render(messages=[{"role": "user", "content": prompt}],bos_token= tokenizer.bos_token,add
    print("-"*80)
    print(f"model_input_string:\n{modelInput}")
    input_ids = tokenizer.encode(modelInput, add_special_tokens=False, return_tensors='pt').to("cuda:0")
    outputs = model.generate(input_ids, do_sample=False)
    model_return_string = tokenizer.decode(*outputs, skip_special_tokens=False)
    print("-"*80)
    print(f"model_return_string:\n{model_return_string}")
    generated_ids = outputs[:, input_ids.shape[1]:]
    generated_text = tokenizer.decode(generated_ids[0], skip_special_tokens=False)
    return generated_text

query = "Please introduce yourself"
print("-"*80)
print(f"query:\n{query}")
response = generate(query)
print("-"*80)
print(f"response:\n{response}")
```

```
/usr/local/lib/python3.11/dist-packages/transformers/generation/configuration_utils.py:628: UserWarning: `do
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/transformers/generation/configuration_utils.py:633: UserWarning: `do
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/transformers/generation/configuration_utils.py:650: UserWarning: `do
  warnings.warn(
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a
--------------------------------------------------------------------------------
query:
Please introduce yourself
--------------------------------------------------------------------------------
model_input_string:
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Please introduce yourself<|im_end|>
<|im_start|>assistant

--------------------------------------------------------------------------------
model_return_string:
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Please introduce yourself<|im_end|>
<|im_start|>assistant
Hello! I'm Qwen, an AI assistant created by Alibaba Cloud. I'm here to help
--------------------------------------------------------------------------------
response:
Hello! I'm Qwen, an AI assistant created by Alibaba Cloud. I'm here to help
```

Data Preparation

Let's load a common dataset, english quotes, to fine tune our model on famous quotes.

```
from datasets import load_dataset

# data = load_dataset("Abirate/english_quotes")
dataset = load_dataset("FreedomIntelligence/Huatuo26M-Lite")
dataset = dataset['train'].map(lambda sample: {"conversations": [{"from": "human", "value": sample['question']},
```

```
README.md: 100%  4.24k/4.24k [00:00<00:00, 14.4kB/s]
format_data.jsonl: 100%  138M/138M [00:06<00:00, 24.6MB/s]
Generating train split: 100%  177703/177703 [00:00<00:00, 511845.22 examples/s]
Map: 100%  177703/177703 [00:16<00:00, 7820.14 examples/s]
```

```
from torch.utils.data import random_split
train_dataset_size, val_dataset_size = 40, 8
train_dataset, val_dataset, _ = random_split(dataset, [train_dataset_size, val_dataset_size, len(dataset)-train_
print(train_dataset[0]['conversations'])
```

```
[{'from': 'human', 'value': '可能是平时比较激动引起的。现在在服用中药和安眠药才能入睡，效果比较慢，已经有半年了不怎么见效。有
```

Customized Dataset

Create a specialized dataset class named "InstructionDataset" designed to handle our custom dataset.

```
import transformers
from typing import Dict, Sequence, List
from torch.utils.data import Dataset
from dataclasses import dataclass

def preprocess(
    sources,
    tokenizer: transformers.PreTrainedTokenizer,
) -> Dict:
    template = Template(tokenizer.chat_template)
    max_seq_len = tokenizer.model_max_length
    messages = []
    for i, source in enumerate(sources):
        if source[0]["from"] != "human":
            # Skip the first one if it is not from human
            source = source[1:]

        for j in range(0, len(source), 2):
            if j+1 >= len(source): continue
            q = source[j]["value"]
            a = source[j+1]["value"]
            assert q is not None and a is not None, f'q:{q} a:{a}'
            input =  template.render(messages=[{"role": "user", "content": q},{"role": "assistant", "content": a
            input_ids = tokenizer.encode(input, add_special_tokens= False)

            query = template.render(messages=[{"role": "user", "content": q}],bos_token=tokenizer.bos_token,add_
            query_ids = tokenizer.encode(query, add_special_tokens= False)

            labels = [-100]*len(query_ids) + input_ids[len(query_ids):]
            assert len(labels) == len(input_ids)
            if len(input_ids) == 0: continue
            messages.append({"input_ids": input_ids[-max_seq_len:], "labels": labels[-max_seq_len:]})

    input_ids = [item["input_ids"] for item in messages]
    labels = [item["labels"] for item in messages]

    max_len = max(len(x) for x in input_ids)

    max_len = min(max_len, max_seq_len)
    input_ids = [ item[:max_len] + [tokenizer.eos_token_id]*(max_len-len(item)) for item in input_ids]
    labels = [ item[:max_len] + [-100]*(max_len-len(item)) for item in labels]

    input_ids = torch.LongTensor(input_ids)
    labels = torch.LongTensor(labels)
    return {
        "input_ids": input_ids,
```

```
            "labels": labels
        }


    class InstructDataset(Dataset):
        def __init__(self, data: Sequence, tokenizer: transformers.PreTrainedTokenizer) -> None:
            super().__init__()
            self.tokenizer = tokenizer
            self.data = data

        def __len__(self):
            return len(self.data)

        def __getitem__(self, index) -> Dict[str, torch.Tensor]:
            sources = self.data[index]
            if isinstance(index, int):
                sources = [sources]
            data_dict = preprocess([e['conversations'] for e in sources], self.tokenizer)
            if isinstance(index, int):
                data_dict = dict(input_ids=data_dict["input_ids"][0], labels=data_dict["labels"][0])
            return data_dict


    @dataclass
    class DataCollatorForSupervisedDataset(object):
        tokenizer: transformers.PreTrainedTokenizer
        def __call__(self, instances: Sequence[Dict]) -> Dict[str, torch.Tensor]:
            input_ids, labels = tuple([instance[key] for instance in instances] for key in ("input_ids", "labels"))
            input_ids = torch.nn.utils.rnn.pad_sequence(
                input_ids,
                batch_first=True,
                padding_value=self.tokenizer.pad_token_id)
            labels = torch.nn.utils.rnn.pad_sequence(labels, batch_first=True, padding_value=IGNORE_INDEX)
            return dict(
                input_ids=input_ids,
                labels=labels,
                attention_mask=input_ids.ne(self.tokenizer.pad_token_id),
            )


    train_dataset = InstructDataset(train_dataset, tokenizer)
    val_dataset = InstructDataset(val_dataset, tokenizer)
    data_collator = DataCollatorForSupervisedDataset(tokenizer=tokenizer)


    sample_data = train_dataset[9]
    IGNORE_INDEX=-100

    print("=" * 80)
    print("Debuging: ")
    print(f"Input_ids\n{sample_data['input_ids']}")
    print(f"Label_ids\n{sample_data['labels']}")
    print("-" * 80)
    print(f"Input:\n{tokenizer.decode(sample_data['input_ids'])}")
    print("-" * 80)
    N_id = tokenizer.encode("N", add_special_tokens= False)[0]
    print(f"Label:\n{tokenizer.decode([N_id if x == -100 else x for x in sample_data['labels']])}")
    print("=" * 80)
```

```
    ================================================================================
    Debuging:
    Input_ids
    tensor([151644,   8948,    198,   2610,    525,   1207,  16948,     11,   3465,
              553,  54364,  14817,     13,   1446,    525,    264,  10950,  17847,
               13, 151645,    198, 151644,    872,    198,  35946,  99486,  99583,
           101913, 105834, 101899, 104719, 105213,   9370,   3837, 108964,  99756,
            43288, 105834,  36993, 109276, 110890,   3837, 105184, 105834,   9370,
           114464, 104719,  11319, 151645,    198, 151644,  77091,    198, 105834,
           101158, 108044,  99252,   3837, 114464, 101097,  32664,  99769, 101899,
             3837,  53153, 100372, 108090,   1773, 105184,   9370, 114464, 100630,
            31843,  43209,  99471,  33108,  47815, 107681,   3837,  99225,  99527,
            49567,  39907,   1773,  18493, 101899, 101925,  85106, 101153, 118945,
             3837, 101171,  26939,  99878, 116771, 100634,  71817, 101899,   1773,
           151645,    198])
    Label_ids
    tensor([  -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,
```

```
                -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,
                -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,
                -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,
                -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100,
                -100,   -100,   -100,   -100,   -100,   -100,   -100,   -100, 105834,
              101158, 108044,  99252,   3837, 114464, 101097,  32664,  99769, 101899,
                3837,  53153, 100372, 108090,   1773, 105184,   9370, 114464, 100630,
               31843,  43209,  99471,  33108,  47815, 107681,   3837,  99225,  99527,
               49567,  39907,   1773,  18493, 101899, 101925,  85106, 101153, 118945,
                3837, 101171,  26939,  99878, 116771, 100634,  71817, 101899,   1773,
              151645,    198])
    ----------------------------------------------------------------------------------
    Input:
    <|im_start|>system
    You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
    <|im_start|>user
    我就是背上的白癜风治疗有哪些要注意的，但是我怕这白癜风会扩散开来，早期白癜风的治疗方法有哪些？<|im_end|>
    <|im_start|>assistant
    白癜风是一种慢性病，治疗方法只能对症治疗，不能完全治愈。早期的治疗方法包括内服药和外用药，光疗等方法。在治疗过程中需要避免劳累，最

    ----------------------------------------------------------------------------------
    Label:
    NNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNNN白癜风是一种慢性病，治疗方法只能对症治疗，不能完全治愈。早期的治疗

    ==================================================================================
```

Training

General Training Hyperparameters

```
# Set training parameters
training_arguments = transformers.TrainingArguments(
    output_dir="./checkpoints",
    num_train_epochs=50,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=1,
    optim='paged_adamw_8bit',
    save_steps=100,
    logging_steps=10,
    learning_rate=1e-5,
    weight_decay=0.001,
    max_steps=-1,
    warmup_ratio=0.1,
    group_by_length=True,
    lr_scheduler_type="cosine",
    gradient_checkpointing=True,
    report_to="none"
)


model.train()
trainer = transformers.Trainer(
    model=model,
    tokenizer=tokenizer,
    args=training_arguments,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    data_collator=data_collator
)
trainer.train()
```

```
<ipython-input-21-7dc37cde72c7>:2: FutureWarning: `tokenizer` is deprecated and will be removed in version 5
  trainer = transformers.Trainer(
No label_names provided for model class `PeftModelForCausalLM`. Since `PeftModel` hides base models input ar
```

[500/500 40:27, Epoch 50/50]

Step	Training Loss
10	2.489500
20	2.518900
30	2.506000
40	2.545400
50	2.509100
60	2.473100
70	2.451300
80	2.433200
90	2.414100
100	2.350600
110	2.285200
120	2.217200
130	2.132900
140	2.024700
150	1.972300
160	1.974300
170	1.967900
180	1.943400
190	1.935200
200	1.923200
210	1.932700
220	1.947500
230	1.928500
240	1.907400
250	1.886100
260	1.903200
270	1.886200
280	1.906300
290	1.912000
300	1.895100
310	1.872500
320	1.903900
330	1.923200
340	1.894900
350	1.871600
360	1.862700
370	1.859100
380	1.850000
390	1.880600
400	1.856700
410	1.848000
420	1.840200

430	1.876000
440	1.855800
450	1.830700
460	1.867100
470	1.841000
480	1.874700
490	1.850900
500	1.875000

```
TrainOutput(global_step=500, training_loss=2.0301459045410155, metrics={'train_runtime': 2433.5438,
'train_samples_per_second': 0.822, 'train_steps_per_second': 0.205, 'total_flos': 1.7499067151204352e+16,
'train_loss': 2.0301459045410155, 'epoch': 50.0})
```

```
def print_trainable_parameters(model):
    """
    Prints the number of trainable parameters in the model.
    """
    trainable_params = 0
    all_param = 0
    for _, param in model.named_parameters():
        all_param += param.numel()
        if param.requires_grad:
            trainable_params += param.numel()
    print(
        f"trainable params: {trainable_params} || all params: {all_param} || trainable%: {100 * trainable_params
    )

model.print_trainable_parameters()
```

```
trainable params: 2,523,136 || all params: 7,618,139,648 || trainable%: 0.0331
```

Once the training is completed, we can evaluate our model and get its perplexity on the validation set like this:

```
import math
!pip install -q -U git+https://github.com/huggingface/accelerate.git
eval_results = trainer.evaluate()
print(f"Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
```



```
[2/2 00:01]
Perplexity: 5.31
```

Save Trained LoRA

```
!pwd
output_path = "ilora"
trainer.save_model(output_path)
```

```
/content
```

Test the trained model

```
template = Template(tokenizer.chat_template)
@torch.no_grad()
def generate(prompt):
    modelInput = template.render(messages=[{"role": "user", "content": prompt}],bos_token= tokenizer.bos_token,a
    input_ids = tokenizer.encode(modelInput, add_special_tokens=False, return_tensors='pt').to("cuda:0")
    outputs = model.generate(input_ids, temperature=1.0)
    model_return_string = tokenizer.decode(*outputs, skip_special_tokens=False)
    print("-"*80)
    print(f"model_return_string:\n{model_return_string}")
```

```
    generated_ids = outputs[:, input_ids.shape[1]:]
    generated_text = tokenizer.decode(generated_ids[0], skip_special_tokens=False)
    return generated_text

query = "我喉咙不太舒服，吃瓜子吃多了，我该怎么办"
print(f"query:\n{query}")
response = generate(query)
print("-"*80)
print(f"response:\n{response}")
```

```
query:
我喉咙不太舒服，吃瓜子吃多了，我该怎么办
--------------------------------------------------------------------------------
model_return_string:
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
我喉咙不太舒服，吃瓜子吃多了，我该怎么办<|im_end|>
<|im_start|>assistant
如果是因为吃太多瓜子导致喉咙不适，可以采取以下措施：

1. 多喝水
--------------------------------------------------------------------------------
response:
如果是因为吃太多瓜子导致喉咙不适，可以采取以下措施：

1. 多喝水
```

Clean GPU Memory

```
# Empty VRAM
# del model
# del trainer
import gc
import torch
torch.cuda.empty_cache()
gc.collect()
gc.collect()
```

```
0
```

```
!nvidia-smi
```

```
Sun Mar 30 16:27:43 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L4                      Off |   00000000:00:03.0 Off |                    0 |
| N/A   76C    P0             34W /   72W |   22283MiB /  23034MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
+-----------------------------------------------------------------------------------------+
```

Load the trained model back and integrate the trained LoRA within.

```
from peft import PeftModel

model = AutoModelForCausalLM.from_pretrained(model_id, load_in_8bit=True, device_map={"":0})
model = PeftModel.from_pretrained(model, output_path)
model = model.merge_and_unload()
model.config.max_length = 512
model.eval()
```

```
tokenizer = transformers.AutoTokenizer.from_pretrained(model_id, padding_side="left")
# tokenizer.pad_token = tokenizer.unk_token
```

```
The `load_in_4bit` and `load_in_8bit` arguments are deprecated and will be removed in the future versions. P
Loading checkpoint shards: 100%  4/4 [00:16<00:00,  4.15s/it]
```

Answer generation

```
@torch.no_grad()
def generate(prompts):
    model_inputs = [template.render(messages=[{"role": "user", "content": prompt}], bos_token=tokenizer.bos_toke
    input_ids = tokenizer(model_inputs, add_special_tokens=False, return_tensors='pt', padding=True).to("cuda:0"

    outputs = model.generate(input_ids.input_ids, attention_mask=input_ids.attention_mask, max_new_tokens=100)

    generated_texts = []
    for i in range(len(prompts)):
        generated_ids = outputs[i, input_ids.input_ids.shape[1]:]
        generated_text = tokenizer.decode(generated_ids, skip_special_tokens=True, clean_up_tokenization_spaces=
        generated_texts.append(generated_text)

    return generated_texts

# test
print("\n\n".join(generate(["我喉咙不太舒服，吃瓜子吃多了，我该怎么办", "Who are you?"])))
```

```
吃瓜子过多可能会导致喉咙不适、干燥或者轻微的炎症。这里有一些建议帮助你缓解不适：

1. **多喝水**：保持身体水分可以帮助缓解喉咙干燥。

2. **避免刺激性食物**：短期内避免辛辣、过热或过冷的食物和饮料，这些都可能加重喉咙的不适感。

3. **适当休息**：给喉咙充分的休息时间有助于恢复。

4. **含片或蜂蜜水**：可以尝试含

I am Qwen, a large language model created by Alibaba Cloud. I am here to assist with a wide range of tasks,
```

Evaluate a trained model on a given test dataset

```
# 删除所有相关对象 (必须放在最前)
if 'model' in globals(): del model
if 'trainer' in globals(): del trainer
if 'optimizer' in globals(): del optimizer

# 强制垃圾回收 (执行两次确保彻底)
import gc
gc.collect()
gc.collect()

# 清空 PyTorch 的 CUDA 缓存
import torch
if torch.cuda.is_available():
    torch.cuda.empty_cache()

# 验证释放结果
print(f"[释放后内存] {torch.cuda.memory_allocated()/1e6:.2f} MB")
```

```
[释放后内存] 12949.49 MB
```

```
!wget https://NLP-course-cuhksz.github.io/Assignments/Assignment1/task1/data/1.exam.json
```

```
--2025-03-30 16:17:48--  https://nlp-course-cuhksz.github.io/Assignments/Assignment1/task1/data/1.exam.json
Resolving nlp-course-cuhksz.github.io (nlp-course-cuhksz.github.io)... 185.199.108.153, 185.199.109.153, 185
Connecting to nlp-course-cuhksz.github.io (nlp-course-cuhksz.github.io)|185.199.108.153|:443... connected.
```

```
HTTP request sent, awaiting response... 200 OK
Length: 86227 (84K) [application/json]
Saving to: '1.exam.json'

1.exam.json         100%[===================>]  84.21K  --.-KB/s    in 0.002s

2025-03-30 16:17:49 (36.0 MB/s) - '1.exam.json' saved [86227/86227]
```

```
import json

with open('1.exam.json') as f:
  data = json.load(f)

print(data[0])
```

```
{'question': '27. 根据国家药品监督管理局，公安部，国家卫生健康委员会的有关规定，又服固体制剂每剂量单位含羟考酮碱不超过5毫克，且
```

```
your_prompt = """请回答下面的多选题，请直接正确答案选项，不要输出其他内容。
{question}
{options}"""

def get_query(da):
  da['options'] = '\n'.join([f"{k}:{v}" for k, v in da['option'].items() if v])
  return your_prompt.format_map(da)

for item in data:
  item['query'] = get_query(item)


print(data[0]['query'])
```

```
请回答下面的多选题，请直接正确答案选项，不要输出其他内容。
27. 根据国家药品监督管理局，公安部，国家卫生健康委员会的有关规定，又服固体制剂每剂量单位含羟考酮碱不超过5毫克，且不含其他麻醉药品
A:含麻醉药品复方制剂的管理
B:第二类精神药品管理
C:第一类精神药品管理
D:医疗用毒性药品管理
```

```
model_answers = generate([item['query'] for item in data])
print(f'\n{model_answers[0]}')
```

```
---------------------------------------------------------------------------
OutOfMemoryError                          Traceback (most recent call last)
<ipython-input-59-3711d4b261b9> in <cell line: 0>()
----> 1 model_answers = generate([item['query'] for item in data])
      2 print(f'\n{model_answers[0]}')

                                 24 frames
/usr/local/lib/python3.11/dist-packages/bitsandbytes/functional.py in int8_mm_dequant(A, row_stats,
col_stats, out, bias)
   2403
   2404     if out is None:
-> 2405         out = torch.empty_like(A, dtype=torch.float16)
   2406
   2407     ptrA = get_ptr(A)

OutOfMemoryError: CUDA out of memory. Tried to allocate 1.43 GiB. GPU 0 has a total capacity of 22.16 GiB of
which 409.38 MiB is free. Process 68616 has 21.76 GiB memory in use. Of the allocated memory 20.51 GiB is
allocated by PyTorch, and 1.00 GiB is reserved by PyTorch but unallocated. If reserved but unallocated
memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See
documentation for Memory Management  (https://pytorch.org/docs/stable/notes/cuda.html#environment-variables)
```

```
import re
from tqdm import tqdm

def get_ans(ans):
    match = re.findall(r'.*?([A-E]+(?:[、 , ]+[A-E]+)*)', ans)
    if match:
        last_match = match[-1]
        return ''.join(re.split(r'[、 , ,]+', last_match))
```

```
        return ''

    correct_num = 0
    total_num = 0
    for model_answer, item in tqdm(zip(model_answers, data)):
      if get_ans(model_answer) == item['answer']:
        correct_num += 1
      total_num += 1
      item['model_answer'] = model_answer

    print(f"ACC: {correct_num/total_num:.2%}")

    result_path = "/content/result.jso"
    with open(result_path, "w", encoding="utf-8") as file:
        json.dump(data, file, ensure_ascii=False, indent=4)
        print(f"Results are save in {result_path}")
```

```
20it [00:00, 110086.72it/s]ACC: 55.00%
Results are save in /content/result.jso
```

```
!zip -r /content/workspace.zip /content/  # 包含所有子目录

from google.colab import files
files.download('/content/workspace.zip')
```